

Lab: 0101

The Day Your Circuit Learns to Remember.

From Combinational to Sequential

How a circuit gains memory — and how a clock keeps that memory sane.

CSA222: Modern Computer Architecture

What You'll Build Today

Three storage elements, in order of increasing discipline. By the end, you'll have the building block of every register on a CPU.

01

SR Latch

Two cross-coupled NORs. Holds one bit. You'll meet its forbidden state — the warning sign of all latches.

02

D Latch

Tame the SR latch with an enable. While $E=1$ the output follows D. While $E=0$ it remembers. Transparent and glitch-prone.

03

D Flip-Flop

Same idea, but it samples only on the clock's rising edge. The workhorse of every synchronous CPU.

Today is where Verilog stops being just combinational equations and starts looking like state.

Recap · Where We Are

Labs 1–4 built circuits whose outputs are pure functions of their inputs. Today the output depends on history too.

01**Combinational so far**

Adders, MUXes, decoders — outputs change the instant inputs change. No state. No clock.

02**But CPUs have state**

Registers hold partial sums between cycles. The PC remembers what to fetch next. Memory remembers everything.

03**Feedback is the answer**

Loop an output back into an input through a gate. With the right topology, the circuit holds onto a bit.

04**Then add discipline**

Pure feedback is messy — async, glitch-sensitive. The clock turns chaos into a sequence of well-defined moments.

Every circuit from today onward will have memory. Get used to thinking in time, not just in logic.

Why We Need Memory

Pure logic forgets the instant inputs change. To compute anything iterative, the circuit has to hold a value between steps.

$f(x)$

what combinational
gates compute

0 ns

how long they
remember

1 bit

the minimum unit
of state we need

∞

complexity we can
build once we have it

THE INSIGHT

Try writing a counter with only AND/OR/XOR. You can't. There's no way to ask 'what was Q a moment ago?' without something that remembers. That something is built from feedback.

Combinational vs Sequential

The single most important distinction in digital design.

COMBINATIONAL

Output = $f(\text{inputs now})$

- No memory — purely reactive.
- Same inputs always give same output.
- Examples: adders, MUXes, decoders.
- The instant inputs change, output follows.

SEQUENTIAL

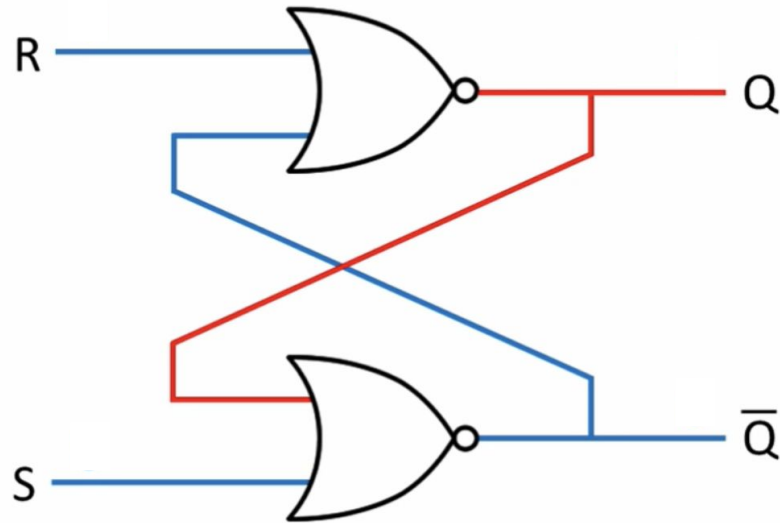
Output = $f(\text{inputs, past state})$

- Has memory — remembers history.
- Same inputs can give different outputs.
- Examples: registers, counters, FSMs.
- A clock decides WHEN state updates.

The difference is one thing: a feedback loop. Add it, and a circuit can remember.

The Idea · Feedback Creates State

Wire an output back into an input. If the topology supports two stable configurations, the circuit can be in either — and stay there.



Each gate's output feeds the OTHER gate's input — that's the cross-coupling that holds state.

WHAT YOU SEE

Two NOR gates feeding each other's inputs. Two stable configurations: (Q=1, Q'=0) and (Q=0, Q'=1). Either holds itself.

SR Latch · Schematic & Truth Table

Two NOR gates, four input combinations, four very different behaviours.

S	R	Q (next)	Behaviour
0	0	Q (hold)	remember
1	0	1	set
0	1	0	reset
1	1	??	forbidden

Read row 4: with S=R=1, both NORs try to drive their outputs to 0 — but then both feedbacks are 0, which would force both back to 1. The latch oscillates.

SR Latch · The Equations

Two assignments, each referring to the other's output. That mutual reference is the storage.

Q (the bit we read)

$$Q = \sim(R \mid Q')$$

Q stays 1 unless R is asserted. If R=1, Q is pulled to 0 regardless of Q'.

Q' (the inverted bit)

$$Q' = \sim(S \mid Q)$$

Q' stays 1 unless S is asserted. If S=1, Q' is pulled to 0, which lets Q rise to 1.

WHY THIS WORKS

When S=R=0 (no command), each equation simplifies to $Q = \sim Q'$, $Q' = \sim Q$. That's a consistent pair — and it's stable. Two such pairs exist; the latch sits in whichever it last entered.

SR Latch · Verilog

Two assigns, each referencing the other's wire. Verilog handles the simultaneous evaluation.

`sr_latch.v`

```
// NOR-based SR Latch
module sr_latch (
    input wire s,
    input wire r,
    output wire q,
    output wire qbar
);
    // Cross-coupled NORs:
    // q depends on qbar, qbar depends on q.
    assign q      = ~(r | qbar);
    assign qbar   = ~(s | q);
endmodule
```

WHY THIS COMPILES

- Each assign is continuous — it re-evaluates whenever its RHS changes.
- When you change s or r, one assign reacts, then the other, then it settles.
- With S=R=1, both keep firing forever — that's the simulator's way of telling you about the forbidden state.
- Real hardware doesn't oscillate cleanly; it lands at whichever NOR is slightly faster.

The Forbidden State · $S = R = 1$

What the truth table calls 'forbidden' isn't an error in Verilog — it's the moment the latch stops being a latch.

1

Both fire

$S=R=1$ forces both NOR outputs to 0. So $Q=0$ and $Q'=0$ — they're equal, breaking the invariant $Q \neq Q'$.

2

Release S and R

Now both inputs return to 0. The NORs feed back the previous 0's — each tries to flip to 1. They race.

3

Whoever is faster wins

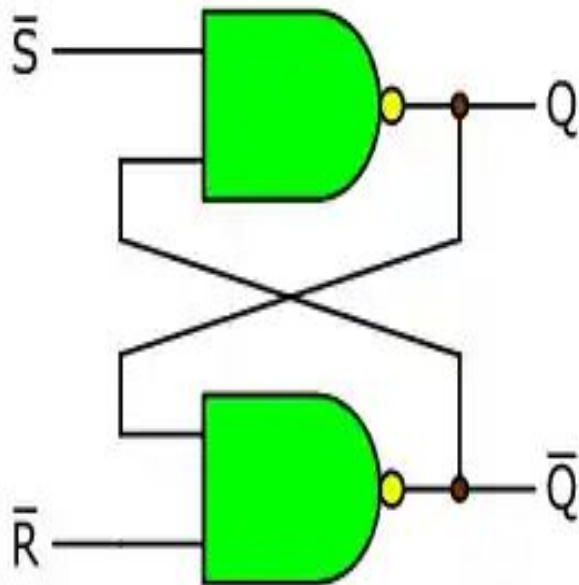
The final state depends on tiny propagation differences between the two gates. Non-deterministic — which is unusable in a CPU.

THE LESSON

Every latch design that exposes both S and R must guarantee they're never both 1. The D latch solves this by construction — R is always $\sim S$.

SR Latch, NAND Version

Same idea, built from NAND gates — but the inputs are active-LOW.



Input		Output	Action
$\bar{S}$	$\bar{R}$	Q_{NEXT}	
1	1	Q	Hold
0	1	1	Set
1	0	0	Reset
0	0	1	Illegal

NOR vs NAND LATCH

NOR latch: active-HIGH (1 = act).

NAND latch: active-LOW (0 = act).

Forbidden state flips too: for the NAND latch, $S' = R' = 0$ is the illegal one.

NAND latches are common in practice because NAND is the cheap, universal gate (recall L2).

Note the primes: S' and R' are active-LOW — a 0 triggers the action, not a 1.

SR Latch · Testbench

Walk through all four cases. The forbidden case will show oscillation in iverilog — read the log carefully.

sr_tb.v

```
module sr_tb;
  reg s, r; wire q, qbar;
  sr_latch dut(.s(s), .r(r), .q(q), .qbar(qbar));
  initial begin
    $dumpfile("sr.vcd"); $dumpvars(0, sr_tb);
    s=0; r=0; #5; // initial - unknown
    s=1; r=0; #5; // SET -> q=1
    s=0; r=0; #5; // HOLD -> q stays 1
    s=0; r=1; #5; // RESET -> q=0
    s=0; r=0; #5; // HOLD -> q stays 0
    s=1; r=1; #5; // FORBIDDEN
    s=0; r=0; #5; // metastable!
  $finish;
end
endmodule
```

WHAT TO LOOK FOR

- Steps 1–5: q follows the rules — set, hold, reset, hold.
- Step 6: q and qbar both go to 0 — illegal state.
- Step 7: after release, iverilog may keep toggling — that's the oscillation.
- If you see X (unknown) on q after step 7, the simulator gave up. That's the right behaviour.

The SR Latch Is Hard to Use

Even when you avoid the forbidden state, two things make raw SR latches dangerous in a real design.

01

Two control lines is one too many

Every consumer of this latch has to remember not to assert S and R together. That's a rule humans break.

02

Any input glitch propagates

A 1 ns spike on S during HOLD will set the latch. There's no way to ignore noise without external gating.

03

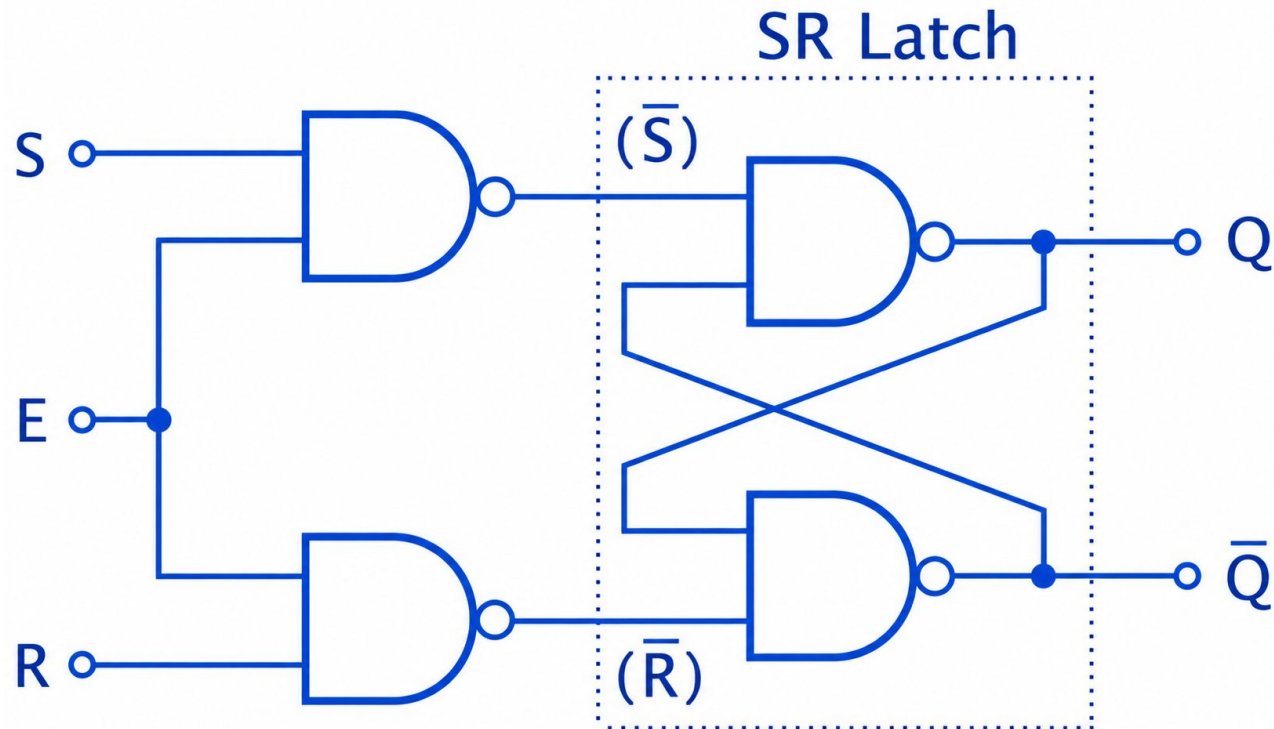
No notion of 'now'

The latch reacts to inputs the instant they change. Synchronous design needs a clock to define when things happen.

The D latch fixes problems 1 and 2. The flip-flop fixes problem 3.

Adding an Enable: The Gated Latch

Now the latch only listens when we say so — the first step toward a clock.



The **Enable (E)** controls the latch: when enabled, **S** sets $Q = 1$ and **R** resets $Q = 0$; when disabled, the previous state is retained.

WHY THIS MATTERS

When $E = 1$ (Enable): The latch responds to **S** and **R** — $S = 1$ sets $Q = 1$, while $R = 1$ resets $Q = 0$.

When $E = 0$ (Disable): The latch **holds its previous state**, so Q remains unchanged.

D Latch · One Input, One Enable

Take the SR latch. Tie $R = \sim S$ internally. Add an enable that gates everything. Now S and R can never both be 1.

$$S = D \cdot E \quad R = \sim D \cdot E$$

01

When $E = 0$

Both S and R are 0. The SR latch is in HOLD. D can wiggle all it wants — output doesn't change.

02

When $E = 1, D = 1$

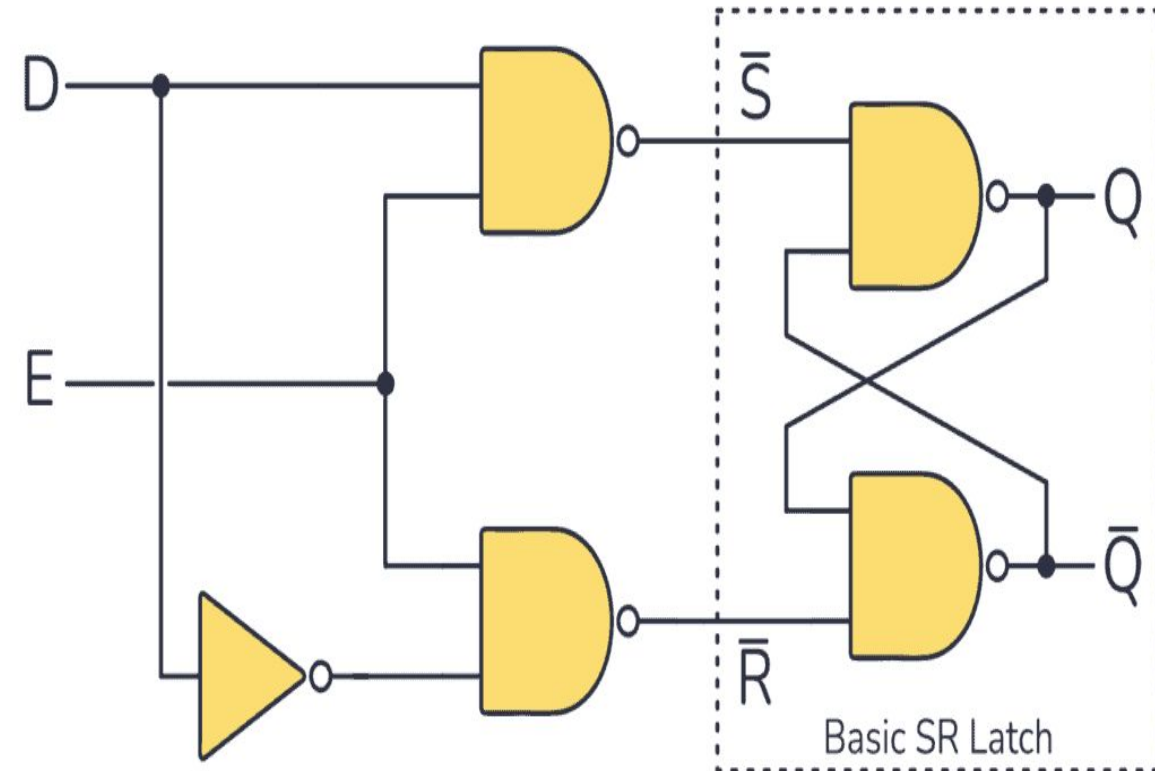
$S=1, R=0$. SR latch is SET. $Q = 1$.

03

When $E = 1, D = 0$

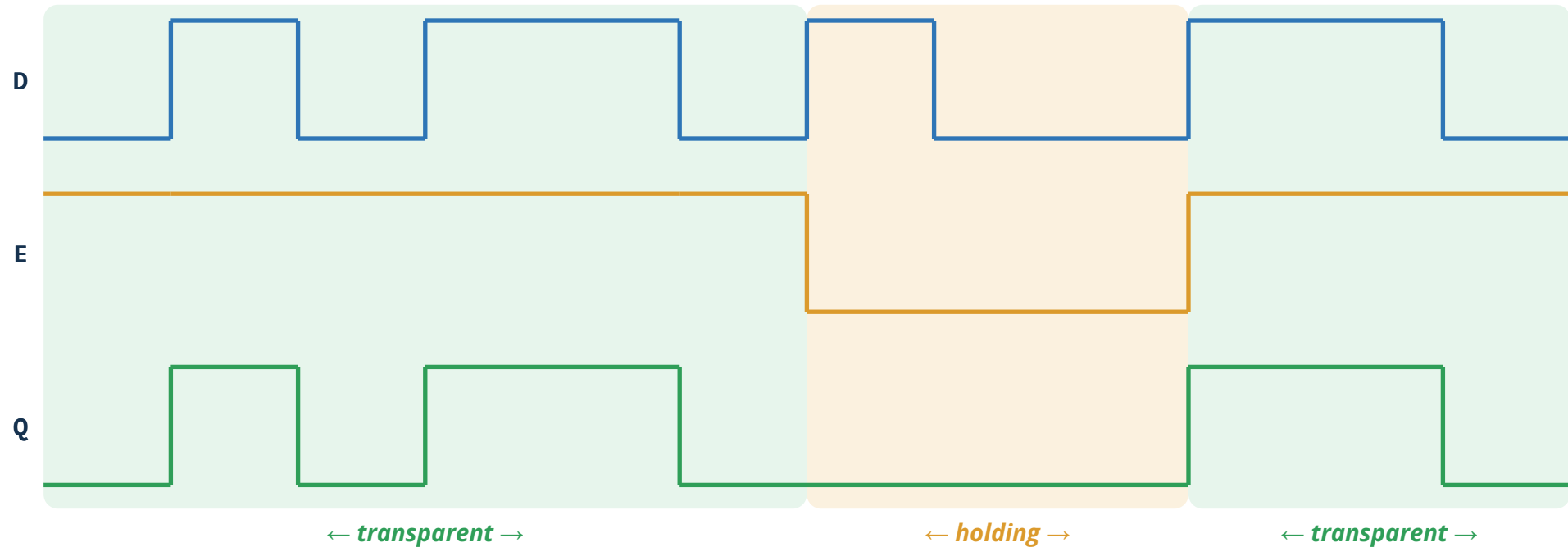
$S=0, R=1$. SR latch is RESET. $Q = 0$.

While E is high, Q follows D — we call this 'transparent'. While E is low, Q holds.



D Latch · Transparency

While E is high, Q tracks D step-for-step. While E is low, whatever D was at the falling edge of E is what Q holds.



During the green windows, Q is a copy of D. During the amber window, Q remembers the last D from before E fell.

D Latch · Verilog

Two assigns to compute S_{int} and R_{int} from D and E . Then reuse the SR latch as a submodule.

d_latch.v

// Transparent D latch built from an SR latch

```
module d_latch (  
    input  wire d,  
    input  wire e,  
    output wire q,  
    output wire qbar  
);  
    wire s_int, r_int;  
    assign s_int = d & e;  
    assign r_int = ~d & e;  
    sr_latch core(.s(s_int), .r(r_int),  
                  .q(q), .qbar(qbar));  
endmodule
```

NOTES

- S and R are mutually exclusive by construction.
- When $E=0$, both s_{int} and r_{int} are 0 — the SR latch holds.
- Reusing `sr_latch` keeps the design hierarchical — like the FA from 2 HAs in Lab 3.
- Compile with both .v files: `iverilog -o d.vvp sr_latch.v d_latch.v d_tb.v`



**Thanks
for
watching!**